

PIPELINE DEVOPS PER 01-flask-base-project

Giuseppe Catillo (Matr. n. 208250)

1. Descrizione del progetto.....	3
Idea e obiettivi.....	3
Fasi della realizzazione.....	3
Tecnologie utilizzate.....	3
2. Containerizzazione.....	4
Dockerfile.....	4
Docker Compose.....	4
3. Script di validazione markdown.....	6
Motivazione.....	6
Implementazione.....	6
4. Pipeline CI/CD.....	7
Struttura della pipeline.....	7
Deployment.....	7
5. Esperimenti eseguiti.....	8
Post errati.....	8
Pipeline.....	8
Persistenza dei dati.....	8
6. Risultati.....	9
7. Possibili sviluppi futuri.....	10

1. Descrizione del progetto

Idea e obiettivi

Il progetto "Strange Blog" consiste in un'applicazione web sviluppata in Python con il framework Flask. L'applicazione funge da blog e da Curriculum Vitae online e si compone di due parti:

- Blog posts: i post sono file Markdown collocati nella cartella *posts/en/* e renderizzati tramite Flask-FlatPages, ognuno dei quali deve rispettare un template specifico comune con campi obbligatori (titolo, autore, data, permalink, ecc.).
- Post views: ogni visualizzazione di un post viene registrata in una tabella *post_views* su un database PostgreSQL, gestita tramite Flask-SQLAlchemy e Flask-Migrate.

Il progetto è stato fornito come base (legacy) su cui applicare le pratiche DevOps.

L'applicazione è volutamente sub-ottimale dal punto di vista strutturale: l'obiettivo, infatti, non è la riscrittura della logica dell'applicazione o l'ottimizzazione del codice sorgente, ma l'applicazione delle metodologie e pratiche DevOps per migliorare l'esperienza dei maintainer, automatizzare il testing, containerizzare l'ambiente e strutturare una pipeline di Continuous Integration e Continuous Deployment (CI/CD).

Fasi della realizzazione

Il lavoro è stato suddiviso in fasi incrementalì:

1. Verifica della baseline in locale: setup del virtual environment ed esecuzione dell'applicazione con un database PostgreSQL per validare il corretto funzionamento delle dipendenze e delle migrazioni del database.
2. Containerizzazione: creazione di un *Dockerfile* multi-stage e di un *.dockerignore*.
3. Orchestrazione: sviluppo di un file *docker-compose.yml* con profili separati per sviluppo (*dev*) e produzione (*prod*), integrando l'applicazione con il database.
4. Validazione dei contenuti: creazione di uno script Bash per validare la conformità dei file Markdown al template richiesto.
5. Automazione CI/CD: implementazione di una pipeline CI/CD su GitLab articolata in fasi di test, build e deploy "simulato" per validare l'intero ciclo di vita del software.

Tecnologie utilizzate

Nel progetto sono state impiegate le seguenti tecnologie principali:

- Python & Flask: linguaggio e framework core dell'applicazione web.
- Flask-FlatPages: parsing e rendering dei file Markdown statici, trattati come record di database.
- Flask-SQLAlchemy: interazione in modo programmatico e sicuro con il database; facilita la gestione delle sessioni.
- Flask-Migrate: gestione delle migrazioni e aggiornamento dello schema del database.
- PostgreSQL: database relazionale per la persistenza del contatore delle visualizzazioni.
- Docker & Docker Compose: containerizzazione dell'applicazione e orchestrazione multi-container (App + DB) in ambienti isolati e riproducibili.
- Bash: linguaggio di scripting utilizzato per la creazione dell'entrypoint del container e dello script di validazione dei post Markdown.
- Unicorn: server WSGI impiegato per gestire le richieste nell'ambiente di produzione in modo concorrente (configurato con 4 worker).
- GitLab CI/CD: piattaforma per l'automazione della pipeline CI/CD (test, build, deploy).

2. Containerizzazione

Una volta predisposto l'ambiente di sviluppo locale (isolato e reso riproducibile) e inizializzato le tabelle del database, seguendo le istruzioni del file *README.md* iniziale, è stato possibile procedere con le attività di DevOps.

Dockerfile

Come prima cosa, l'applicazione è stata containerizzata utilizzando un approccio multi-stage a partire da *python:3.11-slim*, ossia un'immagine leggera (dal *footprint* ridotto rispetto alla versione completa) ma che comprende già tutte le librerie di sistema essenziali per far girare l'applicazione nativamente.

Inoltre, l'ancoraggio a un tag *major.minor* garantisce la stabilità dell'ambiente, permettendo di ricevere automaticamente le patch di sicurezza e gli aggiornamenti di manutenzione (micro-release) ad ogni nuova build, senza incorrere nel rischio di breaking changes.

- Stage Builder: installa le dipendenze in un layer separato copiando inizialmente solo il file *requirements.txt*. Questo permette di sfruttare efficientemente la cache di Docker, evitando di reinstallare le librerie se il codice sorgente cambia.
- Stage Final: copia l'applicazione pre-compilata e le dipendenze. Un aspetto fondamentale per la sicurezza è la creazione esplicita di un utente non-root, limitando i privilegi del processo a runtime. Infine, definisce *entrypoint.sh* come punto di ingresso, con *gunicorn* impostato come comando (CMD) di default per l'ambiente di produzione.

Come accennato, è stato creato un file *entrypoint.sh*, il cui unico scopo è quello di garantire che lo schema del database sia sempre allineato al codice, in totale trasparenza per l'utente, eseguendo automaticamente le migrazioni al lancio del container.

Insieme al Dockerfile è stato configurato un file *.dockerignore*, per evitare di inviare al Docker daemon file non necessari o sensibili durante la fase di build (es. ambiente virtuale *.venv*, cache di Python *__pycache__*, file *.env* contenente le credenziali ecc.).

Questo approccio non solo riduce drasticamente i tempi di build e il peso delle immagini, ma azzerà anche il rischio di far trapelare informazioni sensibili nel Container Registry.

Docker Compose

L'orchestrazione è affidata a *docker-compose.yml*, che sfrutta intelligentemente i profili di Compose per separare gli ambienti. Inoltre, le configurazioni comuni ai servizi web sono state centralizzate sfruttando gli YAML Anchors e richiamate tramite alias.

- Profilo *dev*: avvia il database e un servizio web che sovrascrive il CMD del Dockerfile utilizzando il comando di sviluppo di Flask (*flask run --reload*). Configurando la direttiva *develop.watch*, le modifiche ai file non esclusi dal *.dockerignore* (*action: sync*) vengono sincronizzate nel container istantaneamente senza dover ricostruire l'immagine, mentre le modifiche a *requirements.txt* scatenano un *rebuild* automatico.
- Profilo *prod*: scarica l'immagine pre-compilata dal GitLab Container Registry e utilizza il server *gunicorn*. Integra inoltre la policy *restart: unless-stopped* per garantire la resilienza del servizio e la ripartenza automatica in caso di crash. A questo profilo è stato inoltre aggiunto un *healthcheck* dedicato per monitorare lo stato di salute dell'applicazione, utile durante la successiva fase di deploy. Poiché l'immagine base *python:3.11-slim* è volutamente minimale e priva del comando *curl*, il controllo è implementato sfruttando uno snippet Python inline (*urllib.request.urlopen* sulla porta 8080).

Per evitare una “race condition” tra il container dell’applicazione e quello del database, per cui i servizi *web-X* tentavano di connettersi al database prima che quest’ultimo fosse effettivamente pronto ad accettare connessioni, sono stati utilizzati gli attributi *healthcheck* (su PostgreSQL) + *depends_on: service healthy* (sui servizi web), i quali permettono di descrivere dipendenze tra servizi nativamente nel file *docker-compose.yml*, senza dover installare tool esterni come *netcat* all’interno dell’immagine Docker ed eseguire script di polling TCP. Questo separa in modo pulito l’orchestrazione dalla logica applicativa.

Inizialmente, poi, l’applicazione fornita presentava una configurazione critica nel file *app.py*: l’URI di connessione al database conteneva un indirizzo IP fisso (172.17.0.2) per il database e credenziali in chiaro.

Per rispettare le best practice DevOps, la stringa è stata sostituita con la lettura dinamica di una variabile d’ambiente *'DATABASE_URL'*. In locale, questa variabile viene popolata tramite un file *.env* (ignorato da Git), garantendo la sicurezza delle credenziali e permettendo a Docker Compose di sfruttare la risoluzione DNS interna (puntando correttamente al servizio *db*).

Infine, il volume nominato *flask-postgres-data* garantisce la persistenza dei dati tra i riavvii del container.

3. Script di validazione markdown

Motivazione

Il flusso di pubblicazione prevede che il maintainer aggiunga file Markdown nella cartella *posts/en/*. Se il post è malformato, ossia non conforme al template *posts/template.md.tmpl* (es. campo obbligatorio mancante, formato data errato, separatore assente), l'applicazione Flask potrebbe produrre comportamenti imprevisti solo a runtime.

La soluzione DevOps ideale implementata è quella di intercettare questi errori prima che il codice venga integrato nella codebase, tramite uno script eseguito automaticamente dalla pipeline CI.

Implementazione

Si utilizza, quindi, lo script *scripts/check_markdown_validity.sh* che esamina tutti i post e verifica:

- L'estensione del file, che deve essere necessariamente *.md*.
- La presenza del separatore metadati (---) e che la sezione sovrastante del frontmatter non sia vuota.
- La presenza dei 7 campi obbligatori (*title*, *subtitle*, *author*, *date*, *permalink*, *tags*, *shortcontent*).
- La validità formale e logica della data, rigorosamente interpretabile nel formato %B %d, %Y, attraverso uno snippet Python inline.
- L'assenza di duplicati nei campi dei metadati per evitare sovrascritture impreviste da parte del parser.
- La validità del permalink, che non deve contenere spazi o caratteri di path-traversal (es. /, ., ..).
- La corretta esistenza dei file immagine (*image* e *author_image*), se dichiarati, verificando il percorso nella cartella statica.
- La presenza effettiva del contenuto testuale (il vero e proprio post Markdown) subito dopo il separatore.
- L'unicità globale del permalink su tutti i file analizzati contemporaneamente, per evitare collisioni di routing nell'applicazione Flask.

È stata inoltre introdotta una verifica di corrispondenza tra *filename* e *permalink*.

Sebbene tale vincolo non sia esplicitato nel README originale, è necessario per garantire il corretto funzionamento del routing esistente: la route */blog/<permalink>* utilizza infatti il valore del permalink per costruire il percorso del documento richiesto tramite Flask-FlatPages. In questo modo, un post con permalink *my-first-freelance-exp* deve essere memorizzato come *my-first-freelance-exp.md*.

Lo script utilizza, poi, una variabile *file_errors* per accumulare tutti gli errori prima di restituire un exit code, così da fornire al maintainer un report completo in una singola esecuzione della pipeline.

4. Pipeline CI/CD

Struttura della pipeline

La pipeline in `.gitlab-ci.yml` si articola in tre stage sequenziali:

1. **Test**: esegue `check_markdown_validity.sh` su un'immagine Python pulita. Interrompe la pipeline se rileva post invalidi.
2. **Build**: costruisce l'immagine Docker sfruttando il Docker-in-Docker (DinD) e genera un tag univoco e tracciabile basato sull'hash del commit (`$CI_COMMIT_SHA`). Il job associato, in precise condizioni (push sul branch principale), pubblica anche l'immagine generata sul GitLab Container Registry.
3. **Deploy**: esegue un pull dell'immagine dal registry e avvia i servizi tramite Docker Compose, solo nel caso in cui il job legato allo stage *build* sia andato a buon fine (tramite l'attributo *needs*).

Un aspetto rilevante nella configurazione globale è l'ottimizzazione delle risorse di elaborazione (runner). È stata impostata la direttiva *interruptible*: *true* di default per permettere l'annullamento automatico di pipeline superate da commit successivi.

Inoltre, tramite la configurazione *workflow: rules*, si previene l'esecuzione di pipeline duplicate, evitando l'avvio della *branch pipeline* qualora sia già attiva una pipeline di *Merge Request* per lo stesso ramo.

Per il resto i job sono condizionati:

- Merge Request / Branch secondari: eseguono solo i job di *test* e *build* (senza eseguire il push dell'immagine nel registry), per validare la correttezza formale della modifica prima dell'integrazione.
- Branch *main* (Push): esegue *test*, effettua la *build* e il push nel GitLab Container Registry, sbloccando l'opzione manuale per il *deploy*.

L'autenticazione al GitLab Container Registry avviene in modo nativo e sicuro sfruttando le variabili predefinite iniettate automaticamente dalla CI/CD all'interno dei job (`$CI_REGISTRY_USER`, `$CI_REGISTRY_PASSWORD` e `$CI_REGISTRY`), senza la necessità di configurare o esporre token di accesso esterni.

Deployment

Non essendo prevista un'infrastruttura di produzione dedicata (es. VPS remota o Cluster cloud), il job di deploy è stato implementato come test di validazione finale dell'esecuzione.

Viene utilizzato l'ambiente temporaneo del runner GitLab (in modalità DinD) in cui viene scaricato manualmente il plugin di Docker Compose.

Prima di procedere alla creazione del file `.env`, il job adotta un approccio *fail-fast*, verificando l'effettiva presenza delle variabili d'ambiente segrete della CI interrompendo immediatamente l'esecuzione in caso di anomalie.

Infine, per la verifica finale, invece di affidarsi a una singola richiesta HTTP esterna con *curl*, che in un ambiente CI fallirebbe a causa dell'isolamento di rete intrinseco tra il *job container* e il demone *Docker-in-Docker* che espone effettivamente le porte, il job sfrutta la comunicazione diretta con il demone tramite `$DOCKER_HOST`.

Esegue quindi un loop di polling che interroga lo stato interno del container *flask-prod* tramite il comando *docker inspect*. Lo script attende attivamente che il servizio transiti nello stato *healthy* (basandosi sull'healthcheck Python configurato nel Compose); in caso contrario, il job fallisce stampando automaticamente gli ultimi 200 log del container per facilitare il debugging.

5. Esperimenti eseguiti

Post errati

Per verificare il corretto funzionamento dello script, è stato creato un post Markdown intenzionalmente invalido e inserito nel repository. Il file è stato costruito in modo da generare la maggior parte degli errori previsti dal validator, verificando in particolare la gestione simultanea di più errori relativi allo stesso post.

Grazie all'implementazione della variabile `file_errors`, lo script raccoglie e stampa messaggi esplicativi di tutti gli errori relativi a ciascun file in un unico log, senza interrompersi al primo errore. La variabile `EXIT_CODE` mantiene invece il risultato complessivo della validazione, restituendo exit code 1 se è stato rilevato almeno un errore.

In questo modo, la pipeline CI ha bloccato lo stage di `test` della pipeline, impedendo alla `build` di proseguire, come atteso.

Per completezza, l'inserimento di post formattati in modo rigoroso ha verificato con successo lo sblocco e il corretto scorrimento dell'intera pipeline.

Pipeline

Dopo aver configurato le GitLab Variables, è stato testato l'intero flusso della CI/CD.

La sequenza di eventi osservata è stata:

1. Aprendo una MR verso il branch `main`, la pipeline si è attivata eseguendo esclusivamente i job `test-job` e `build-job`. Si è potuto verificare con successo che l'immagine venisse compilata correttamente, ma che non venisse effettuato il push nel Container Registry, confermando l'ottimizzazione delle risorse.
2. Una volta approvata la MR, l'operazione di merge ha innescato automaticamente la pipeline sul branch principale.
3. Il `build-job`, rilevando di trovarsi ora su `main`, ha costruito l'immagine, si è autenticato e ha completato con successo il push sul GitLab Container Registry.
4. Sbloccatosi in stato di attesa, è stato avviato manualmente il job `deploy-job`: il runner (tramite DinD) ha installato il plugin Compose, creato il file `.env` dalle variabili CI, scaricato la nuova immagine con `docker compose pull` e avviato lo stack.
Infine, all'interno dello stesso job, lo smoke test ha eseguito il polling dello stato del container. Non appena l'healthcheck interno ha confermato la disponibilità dell'app, il loop ha rilevato lo stato `healthy`, validando il corretto avvio e terminando la pipeline con successo.

Persistenza dei dati

Per verificare la tenuta del database, lo stack è stato avviato e sono stati aperti alcuni post per incrementare il contatore delle visualizzazioni.

Arrestando l'ambiente con `docker compose --profile dev down` e riavviandolo successivamente, i dati del database sono risultati intatti grazie all'utilizzo del named volume `db-data` montato su `/var/lib/postgresql/data/`.

6. Risultati

Un tipico flusso di lavoro del maintainer si presenta come segue:

1. Il maintainer aggiunge un nuovo file Markdown in `posts/en/`.
2. Crea un branch dedicato, effettua il commit e apre una Merge Request verso il branch *main*.
3. La pipeline CI si attiva automaticamente per la Merge Request, eseguendo esclusivamente gli stage di *test* e *build*:
 - a. Se il post è invalido → lo stage di test fallisce, bloccando l'approvazione della Merge Request.
 - b. Se il post è valido → la pipeline passa, rendendo la Merge Request approvabile in totale sicurezza.
4. Al merge su *main*, la pipeline si riattiva sul ramo principale: esegue nuovamente i test, compila l'immagine Docker definitiva, ne effettua il push sul GitLab Container Registry e sblocca il job di *deploy*.

Quest'ultimo non viene eseguito in automatico ma richiede un'approvazione manuale (essendo configurato con l'attributo *when: manual*), garantendo il controllo umano sull'effettivo rilascio in esecuzione.

Il progetto presenta ora un ecosistema DevOps pienamente funzionale:

1. Immagini Ottimizzate e Sicure: il *Dockerfile* multi-stage produce un'immagine compatta e sicura, eseguita senza privilegi di root e che sfrutta efficientemente la cache per l'installazione delle dipendenze. In questo modo è possibile avviare l'app in qualsiasi ambiente tramite Docker Compose.
2. Pipeline CI/CD Robusta: Il file *.gitlab-ci.yml* definisce un flusso coerente in tre stadi: *test*, *build* e *deploy*.
 - Sulle Merge Request e sui branch secondari vengono eseguiti solo i test (validazione markdown) e la build per validare la correttezza formale.
 - Solo in caso di push sul branch *main*, l'immagine generata viene pubblicata nel GitLab Container Registry, "taggata" con l'hash univoco del commit (*\$CI_COMMIT_SHA*) per garantirne l'immutabilità e la tracciabilità.
3. Manutenibilità Elevata: Il maintainer non deve più preoccuparsi della configurazione dell'ambiente o delle migrazioni del DB, operazioni ora gestite in totale trasparenza dai container e dall'entrypoint. L'inserimento di post errati, poi, viene bloccato a monte dalla pipeline.

7. Possibili sviluppi futuri

- La fase di *deploy* nella pipeline avviene localmente e in modo manuale all'interno del job GitLab (attraverso *Docker-in-Docker*) al solo scopo di validare l'effettiva eseguibilità dell'immagine.

Effettuare il deploy automatico su un'infrastruttura dedicata assegnata, come una Virtual Machine remota (es. AWS EC2) o più nodi worker di un cluster Docker Swarm/Kubernetes, consentirebbe di eliminare il single point of failure attuale e ottenere un'alta disponibilità, oltre che funzionalità più avanzate di scheduling e autoscaling (in concomitanza con una maggiore complessità operativa).

Al tempo stesso, si otterrebbe la possibilità di effettuare aggiornamenti senza alcun downtime.

La rimozione, poi, del vincolo di avvio manuale del deploy (trasformandolo in un job automatico) completerebbe la transizione da un modello di Continuous Delivery a uno di vera Continuous Deployment.

- Lo script di testing copre solo la validazione dei file Markdown.

Introdurre una fase di analisi statica del codice (linting Python via *Flake8*, ad esempio) e test unitari scritti in Python garantirebbe la correttezza generale del prodotto e maggiore stabilità alle funzionalità core dell'applicazione legacy (es. routing, logica DB, risposte HTTP).

- In previsione futura, si potrebbe pensare ad una separazione più netta di quelli che sono i file di configurazione ed un utilizzo maggiore delle funzionalità di caching messe a disposizione da GitLab CI (es. salvando la cartella *.cache/pip* tra i job) o all'utilizzo di pacchetti Python precompilati (Wheels) creati in uno stage precedente, per velocizzare ulteriormente i tempi della fase di build.
- Si potrebbe pensare di integrare nello stage di *test* della CI/CD dei tool di scansione per individuare vulnerabilità (CVE) note all'interno dell'immagine Docker base o tra le dipendenze dichiarate in *requirements.txt*.

Inoltre, pur eseguendo già il container con un utente non-root, l'hardening potrebbe essere incrementato forzando il file system del container in modalità *read-only* (ad eccezione di specifiche cartelle temporanee).

Infine, per ambienti aziendali, si potrebbe demandare la gestione delle password del database a un Secret Manager esterno, superando l'attuale iniezione tramite le variabili d'ambiente standard della CI.